

PROJECT REPORT

Revolutionizing Cancer Detection: A Machine Learning Approach

Anurag Mishra

Dhandeep Singh

DECLARATION

We hereby declare that this written submission represents our own ideas in our own words and where others' ideas or words have been included, have been adequately cited and referenced the original sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission.

Date: 21 November 2024

Dhandeep Singh
Anurag Mishra

ABSTRACT

Road accidents occurring during the past couple of years at an alarming rate particularly due to the two-wheelers index, therefore, reflects the demand for strong solutions to work. Saves lives through timely provision of medical services. Response to emergencies is delayed: causes heavy losses to patients involved in accidents. For this reason, we Design an automated accident detection system and response mechanism using Arduino. Uno, accelerometer, GSM module and GPS to get emergency response very quickly within case of accidents. The accelerometer sensor carries out the fundamental working of the system by continuously monitoring the tilt of the vehicle and even violent movements that might occur due to the accident. Every time such movement crosses the threshold limit, Arduino Uno verifies the sensor readings so that it ascertains that there is an accident indeed. Once it becomes active, the GSM module generates alert messages with all the important information. One of the main characteristics of the system is that it contains the GPS module. It captures the coordinates of the place of accident very accurately. The system sends those coordinates instantly over SMS to emergency contact numbers, neighboring medic facilities, and law enforcement organizations. This process allows to promptly understand the place of accident and therefore decreases the time intervals of the reaction and increases the chances of survival of accident victims. It is combined with a friendly application for Android to make the system accessible easily. The application forwards actual coordinates to emergency contacts, and how to get to that location is assisted by their mapping services. It also allows canceling false alerts within specified time, thus not dispensing unnecessary services. Conclusion: The proposed system thus offers an effective scalable framework for real-time accident detection with automatically generated notices of emergency using accessible affordable components. Such a system has the potential to cause considerable safety factors in roads as well as to minimize fatalities, as proper mediations would result from prompt responses.

Paper Citation:

S. Costanzo, A. Flores and G. Buonanno, "Fast and Accurate CNN-Based Machine Learning Approach for Microwave Medical Imaging in Cancer Detection," in IEEE Access, vol. 11, pp. 66063-66075, 2023, doi: 10.1109/ACCESS.2023.3291076.

keywords: {Convolutional neural networks;Image reconstruction;Breast;Computer architecture;Tumors;Dielectrics;Computational modeling;Scattering;Machine learning;Iterative methods;Image segmentation;Inverse scattering problem;machine learning;Born iterative method;convolutional neural networks;breast cancer;tumor segmentation}

ACKNOWLEDGEMENT

We have taken efforts in this project. However, it would not have been possible without the kind support and help of many individuals. We would like to extend our sincere thanks to all of them. It has been a privilege to work on this project. We would like to place on record our deep sense of gratitude to Dr. Ritu Raj and Mr. Ritesh Kumar Sharma for their generous guidance and constant supervision as well as for providing necessary information regarding the project and also for their support in completing the project. Their constant guidance and willingness to share their vast knowledge made us understand this project and its manifestations in great depths and helped us to complete the assigned tasks on time. We also wish to extend our thanks to our classmates for their insightful comments and constructive suggestions to improve the quality of this project work.

SIGNATURE OF GROUP MEMBERS:

Anurag Mishra

Dhandeep Singh

Date:16 November 2024

TABLE OF CONTENTS

CERTIFICATE	i
DECLARATION	ii
ABSTRACT	iii
ACKNOWLEDGEMENT	iv
TABLE OF CONTENTS	v
CHAPTER 1 INTRODUCTION	1
1.1 Introduction	1
CHAPTER 2 SOLVING ISP	5
CHAPTER 3 METHODOLOGY.....	10
CHAPTER 4 CNN.....	16
CHAPTER 5 SEGMENTATION.....	20
CHAPTER 6 SCOPE.....	25
CONCLUSION.....	26
REFERENCES	27
APPENDIX A	30

CHAPTER 1 INTRODUCTION

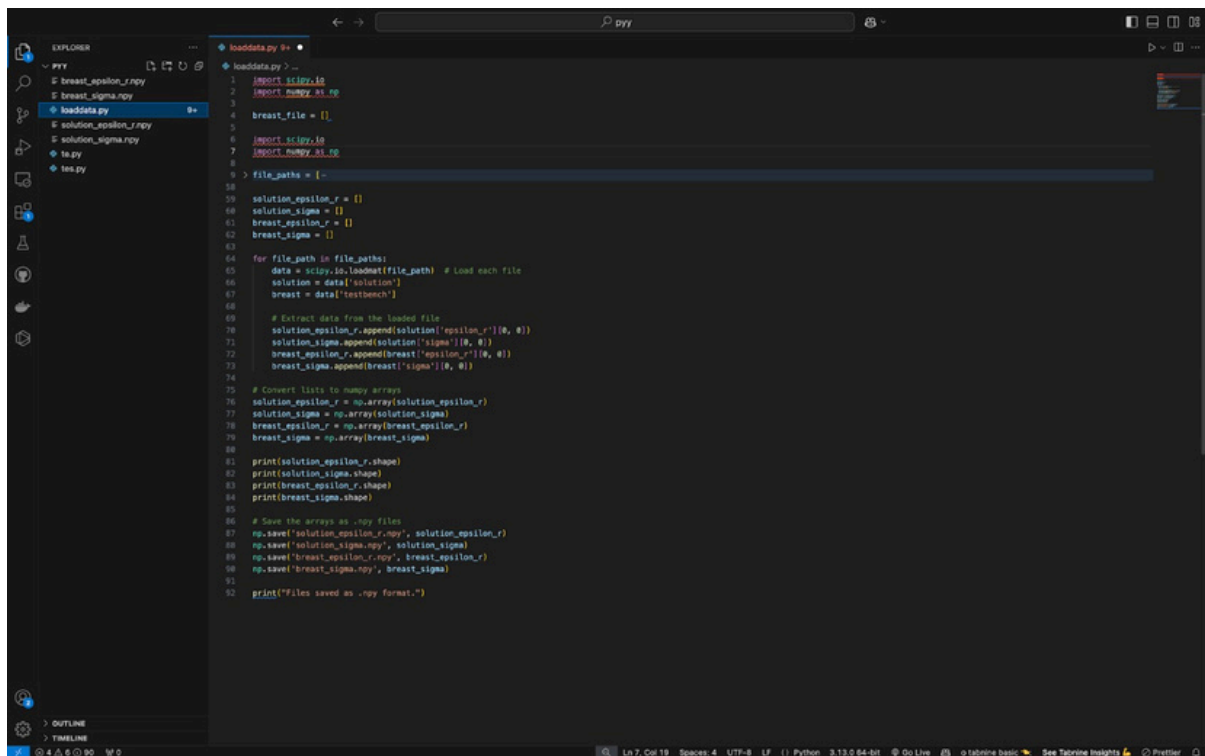
1.1 Introduction

Inverse Scattering Problems (ISPs) play a crucial role in microwave imaging techniques, helping identify a target's size, position, shape, and properties like relative permittivity and conductivity by analyzing the scattered field. These problems form the foundation for various applications, including biomedical imaging, through-the-wall imaging, remote sensing, and geophysics. However, ISPs often face challenges due to their nonlinear nature and ill-posedness. Traditional methods, such as the distorted Born iterative method (DBIM), contrast source inversion (CSI), and subspace optimization methods, use nonlinear optimization with regularization to estimate unknown parameters. While these approaches are powerful and widely applied, they come with high computational costs, and their accuracy can degrade significantly when dealing with strong scatterers or severe nonlinearities.

To overcome these limitations, machine learning techniques, especially deep convolutional neural networks (CNNs), have emerged as promising solutions. CNNs leverage large datasets and advanced computational resources to address inverse problems effectively. These networks have shown success in fields like X-ray computed tomography, computational optical imaging, and magnetic resonance imaging. For example, CNN-based methods like DeepNIS combine traditional iterative techniques with modern neural network architectures to enhance precision in dielectric imaging. Other approaches, such as U-NET and ResNet, further push the boundaries of accuracy and efficiency in solving ISPs by integrating skip connections and residual learning.

Despite their potential, training deep neural networks demands significant computational resources and large datasets. Balancing computational efficiency with reconstruction accuracy involves optimizing network architecture, tuning hyperparameters, ensuring sufficient training data, and managing the cost of training.

This work introduces a framework that integrates the Born Iterative Method (BIM) with quadratic programming and CNN-based techniques for image reconstruction and segmentation. Specifically, it evaluates three architectures: a U-Net for reconstructing breast dielectric profiles and a U-Net combined with ResNet18 for tumor segmentation. The study compares these approaches using breast model data from the University of Wisconsin Computational Electromagnetics Laboratory (UWCEM) to determine the most effective model. Metrics such as execution time for quadratic BIM, training time per epoch, total computation time, and post-training reconstruction time are analyzed to identify an optimal solution that balances performance and efficiency.



```
1 import numpy as np
2 import scipy.io
3
4 breast_file = []
5
6 import scipy.io
7 import numpy as np
8
9 file_paths = [
10
11
12
13
14
15
16 solution_epsilon_r = []
17 solution_sigma = []
18 breast_epsilon_r = []
19 breast_sigma = []
20
21 for file_path in file_paths:
22     data = scipy.io.loadmat(file_path) # Load each file
23     solution = data['solution']
24     breast = data['breast']
25
26 # Extract data from the loaded file
27 solution_epsilon_r.append(solution['epsilon_r'][0, 0])
28 solution_sigma.append(solution['sigma'][0, 0])
29 breast_epsilon_r.append(breast['epsilon_r'][0, 0])
30 breast_sigma.append(breast['sigma'][0, 0])
31
32 # Convert lists to numpy arrays
33 solution_epsilon_r = np.array(solution_epsilon_r)
34 solution_sigma = np.array(solution_sigma)
35 breast_epsilon_r = np.array(breast_epsilon_r)
36 breast_sigma = np.array(breast_sigma)
37
38 print(solution_epsilon_r.shape)
39 print(solution_sigma.shape)
40 print(breast_epsilon_r.shape)
41 print(breast_sigma.shape)
42
43 # Save the arrays as .npy files
44 np.save('solution_epsilon_r.npy', solution_epsilon_r)
45 np.save('solution_sigma.npy', solution_sigma)
46 np.save('breast_epsilon_r.npy', breast_epsilon_r)
47 np.save('breast_sigma.npy', breast_sigma)
48
49 print("Files saved as .npy format.")
```

1. Overview of Microwave Imaging in Medical Applications

Microwave imaging has emerged as a powerful non-invasive diagnostic tool in biomedical engineering. It leverages electromagnetic waves to reconstruct internal structures, enabling clinicians to identify and characterize abnormalities. One of its most promising applications lies in breast cancer detection, offering a low-risk alternative to traditional imaging techniques.

2. Breast Cancer Detection: A Persistent Challenge

Globally, breast cancer remains one of the leading causes of cancer-related deaths among women. Early and accurate detection is pivotal to improving survival rates. Despite advances in mammography, MRI, and ultrasound, challenges such as high costs, radiation exposure, and limited sensitivity in dense tissues persist.

3. The Need for Advanced Imaging Techniques

Conventional imaging modalities often struggle with limitations in sensitivity, especially for dense breast tissues. Microwave imaging, with its ability to differentiate between tissues based on their dielectric properties, provides a promising solution. This approach can enhance detection accuracy, especially for small or early-stage tumors.

4. Role of Inverse Scattering Problems (ISP)

At the core of microwave imaging lies the inverse scattering problem (ISP), which reconstructs

the dielectric properties of tissues from measured electromagnetic fields. ISPs are inherently nonlinear and ill-posed, requiring sophisticated algorithms to ensure accurate reconstructions.

5. Computational Challenges in ISPs

Traditional approaches to solving ISPs, such as the Born Iterative Method (BIM), are computationally intensive and sensitive to noise. They struggle with convergence when handling strong scatterers, such as tumors, and often fail to achieve high-resolution reconstructions in practical scenarios.

6. Machine Learning: Transforming Medical Imaging

Recent breakthroughs in machine learning have transformed numerous fields, including medical imaging. Deep learning models, particularly convolutional neural networks (CNNs), excel at feature extraction and pattern recognition, making them ideal candidates for enhancing ISP solutions.

7. CNNs and Biomedical Imaging

Convolutional neural networks (CNNs) have demonstrated remarkable success in various imaging applications, from X-ray and MRI reconstruction to tumor segmentation. Their ability to learn from large datasets and generalize to unseen scenarios makes them invaluable for biomedical research.

8. Combining BIM and CNN for Microwave Imaging

This report explores an innovative hybrid approach that integrates BIM with CNN architectures, specifically U-Net and ResNet, to overcome the limitations of traditional ISP methods. By doing so, it aims to achieve faster and more accurate reconstructions while reducing computational overhead.

9. The UWCEM Breast Phantom Dataset

To validate the proposed approach, data from the UWCEM breast phantom repository is utilized. This dataset offers anatomically realistic MRI-based breast models with varying levels of tissue density, ensuring robust testing across diverse scenarios.

10. Addressing Strong Scatterers in Breast Imaging

Tumors, being strong scatterers, introduce significant nonlinearity into the imaging process.

The hybrid approach proposed in this work incorporates machine learning as a regularization technique, mitigating these challenges and ensuring precise reconstructions.

11. Tumor Segmentation with U-Net and ResNet

Segmentation is critical for identifying tumor boundaries accurately. The integration of U-Net with ResNet, leveraging residual connections, enhances segmentation precision. This synergy ensures that even minute tumor regions are effectively detected.

12. Noise Resilience in Real-World Scenarios

Real-world imaging scenarios are often noisy. This report evaluates the performance of the proposed approach under varying noise levels, including 10 dB and 20 dB Gaussian noise, highlighting its robustness and practical applicability.

13. Evaluating Computational Efficiency

Beyond accuracy, computational efficiency is a critical consideration. By optimizing CNN architectures and leveraging pre-trained weights, the proposed method achieves significant reductions in training and inference times compared to traditional approaches.

14. Broader Implications for Medical Imaging

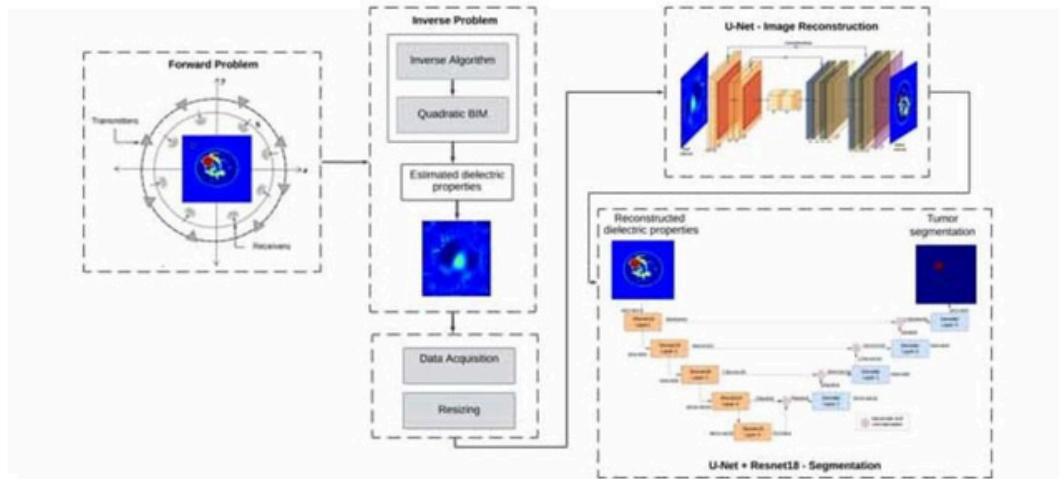
While focused on breast cancer detection, the methods and insights discussed in this report have broader implications. They can be extended to other medical imaging applications, such as brain imaging or through-the-wall imaging, demonstrating the versatility of the approach.

15. Objectives of the Report

This report aims to systematically analyze the integration of BIM and CNN for breast cancer detection. It evaluates the accuracy, robustness, and computational efficiency of different network architectures, providing a comprehensive framework for future research.

16. Roadmap of the Report

The subsequent sections delve into the methodology, numerical results, and conclusions. They provide detailed insights into the dataset, experimental setup, and outcomes, ensuring a clear understanding of the proposed approach and its potential to transform microwave imaging.



U-Net CNNs are applied to guarantee a fast and accurate convergence of inverse scattering problems solved by Born iterative method, even in the presence of strong scatterers. A novel approach is also introduced which integrates U-Net and Resnet models to realize a segmentation process, thus leading to the effective feature extraction and the accurate identification of anomalies within healthy tissue. [View less](#)

CHAPTER 2 : ISP PROBLEM

Inverse Scattering Problem

The scattering process recovers the spatial distribution of dielectric parameters. This procedure is executed by employing three MRI (Magnetic Resonance Imaging) heterogeneous numerical breast phantoms (Class 1, Class 2, Class 3) from the available Repository of the University of Wisconsin Computational Electromagnetics Laboratory(UWCEM) [29]. This online repository contains a collection of anatomically realistic MRI-based numerical breast phantoms which incorporate the realistic dispersive dielectric characteristics of normal breast tissue between 0.5 and 20 GHz. In the present work, a single 1GHz frequency is adopted. BIM with quadratic programming is applied to solve the forward problem with 11 transmitting antennas and 11 receiving antennas, as shown in Fig. 1. The diameter of the acquisition circle is equal to 20 cm. The background medium permittivity is fixed to 10, with zero conductivity (ideal lossless medium), while breast phantoms have a realistic range of permittivity between a minimum of 2.5 and a maximum of 67. As demonstrated in our previous works [26], [27], [28], it is not possible to obtain a sufficient and accurate reconstruction using the quadratic BIM model only. In order to overcome this issue, a data collection containing simulations of the various breast

phantoms is created by solving the ISP. Therefore, the technique based on the BIM model and quadratic programming seeks to obtain as many samples as possible, including the permittivity and conductivity distribution of the breast models. Each sample consists of a permittivity and conductivity distribution vector with a 128×128 element resolution. A total of 1500 samples of three types of phantoms (from the least dense to the most dense one) are simulated by introducing tumors in random positions inside the breast tissue with diameters between 0.6 cm and 1.6 cm. The distribution and quantity of samples are depicted in Fig. 2. Subsequently, all samples are separated into a training set and a validation set, to be trained in the convolutional neural network stage.

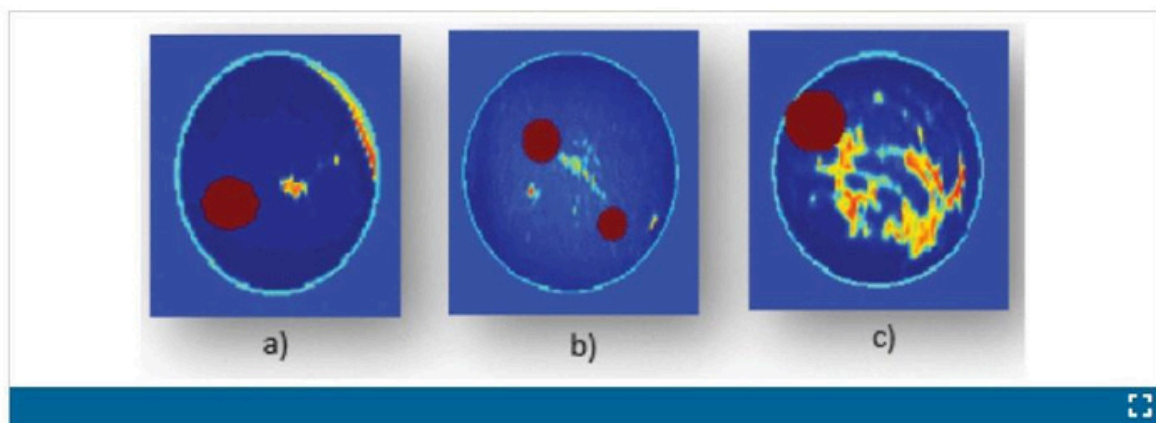


FIGURE 2. Breast phantoms and number of different samples. a) Class 1: 500 samples, b) Class 2: 500 samples and c) Class 3: 500 samples.

Understanding Inverse Scattering Problems (ISP)

1. Definition and Importance

Inverse Scattering Problems (ISP) are fundamental mathematical formulations aimed at reconstructing unknown properties of a medium or object based on the interaction of waves with it. This process involves deducing the size, shape, position, and material properties of the object from the scattered wavefield measurements. ISPs are crucial in many fields, including biomedical imaging, geophysics, nondestructive testing, and remote sensing. In the biomedical domain, ISPs form the backbone of microwave imaging systems, enabling the detection and characterization of anomalies such as tumors.

2. Principles of Scattering

When an electromagnetic wave propagates through a medium, it interacts with the material's

internal structures, leading to scattering. The nature of this scattering is influenced by the medium's electromagnetic properties, primarily its permittivity and conductivity. The scattered field carries encoded information about these properties, which ISPs aim to decode.

3. The Forward and Inverse Problems

ISPs are typically divided into two components:

- **Forward Problem:** Predicting the scattered field based on known medium properties and wave interactions. This step uses Maxwell's equations to model how electromagnetic waves propagate through the medium. The forward problem is generally well-posed, meaning it has a unique solution that depends continuously on the input.
- **Inverse Problem:** Reconstructing the medium's properties from the measured scattered field. This step is inherently ill-posed, as the solution may not be unique, and small errors in the input (measurement noise) can lead to large deviations in the output.

4. Challenges in Solving ISPs

The ill-posed nature of ISPs introduces several challenges:

- **Nonlinearity:** The relationship between the scattered field and the object's properties is nonlinear, especially in high-contrast scenarios like biological tissues.
- **Computational Intensity:** The iterative algorithms required to solve ISPs often involve high computational costs due to the large number of unknowns and the need for repeated forward problem solutions.
- **Noise Sensitivity:** Measurement errors or environmental noise can significantly impact the accuracy of the reconstruction.

5. Applications in Biomedical Imaging

In the context of biomedical imaging, ISPs are used to reconstruct the dielectric properties of tissues, which vary between healthy and abnormal regions. For example, cancerous tissues have higher permittivity and conductivity compared to normal tissues. This contrast forms the basis for tumor detection using microwave imaging. By solving the ISP, clinicians can obtain high-resolution maps of the internal tissue structure, aiding in early diagnosis.

6. Traditional Approaches to Solving ISPs

Several conventional algorithms have been developed to address ISPs, each with its strengths and limitations:

- **Born and Distorted Born Iterative Methods (BIM/DBIM):** These linearization techniques assume weak or moderate scatterers and iteratively refine the solution. While computationally efficient, their accuracy diminishes for high-contrast scatterers.

- Contrast Source Inversion (CSI): This method reformulates the ISP by focusing on the contrast source, achieving better convergence for complex scenarios. However, it remains computationally expensive.
- Subspace Optimization Methods (SOM): These methods leverage reduced-dimensional subspaces to optimize the inversion process, balancing computational cost and accuracy.

7. Limitations of Traditional Methods

Despite their utility, traditional ISP techniques face significant challenges in real-world biomedical applications:

- Poor Convergence: In cases with strong scatterers, such as dense tumors, conventional methods often fail to converge or provide accurate reconstructions.
- High Computational Load: The iterative nature of these methods requires significant computational resources, limiting their real-time applicability.
- Dependence on Initial Guesses: The quality of reconstruction is highly sensitive to the initial assumptions, making it difficult to achieve reliable results in heterogeneous media.

8. Advancements Through Machine Learning

Recent advancements in machine learning have introduced promising solutions to the challenges of ISPs. Deep learning models, particularly convolutional neural networks (CNNs), excel at handling nonlinearity and extracting features from complex datasets. By integrating machine learning with traditional ISP methods, it becomes possible to:

- Accelerate convergence by learning the underlying relationships between the scattered field and medium properties.
- Enhance robustness to noise through data-driven training on diverse datasets.
- Reduce computational costs by leveraging pre-trained networks and GPU acceleration.

9. Hybrid Approaches: A Transformative Shift

One effective hybrid approach involves combining the Born Iterative Method (BIM) with CNN architectures. BIM efficiently solves the forward problem and generates preliminary reconstructions, which are then refined using CNNs. This combination ensures a balance between computational efficiency and reconstruction accuracy, making it particularly suitable for biomedical imaging scenarios.

10. ISP in Breast Cancer Detection

In breast cancer detection, ISPs enable the reconstruction of dielectric profiles of breast tissues from measured microwave signals. Tumors, being strong scatterers, introduce high contrast in permittivity and conductivity. By solving the ISP, clinicians can accurately localize and

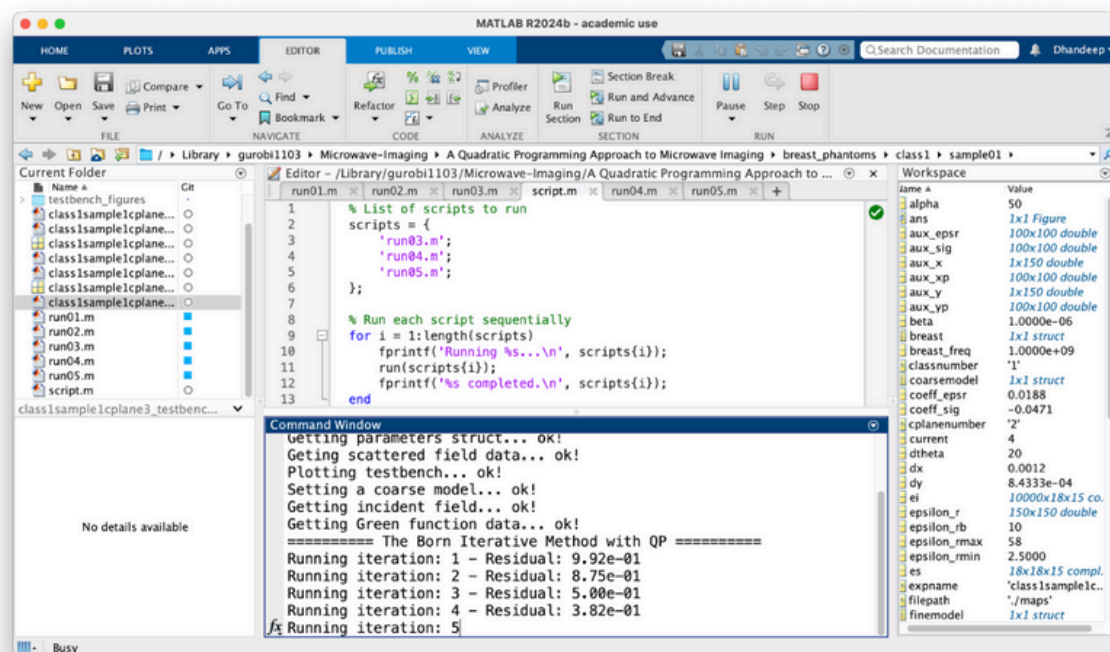
characterize these anomalies, even in dense breast tissues. The use of machine learning-enhanced ISP methods further ensures reliable reconstructions in noisy environments.

11. The Role of the UWCEM Breast Phantom Dataset

To validate ISP solutions, datasets like the UWCEM breast phantom repository are invaluable. These datasets provide realistic numerical models of breast tissues with varying densities and embedded anomalies, enabling robust testing of ISP methodologies. Using such datasets ensures that ISP solutions are applicable to real-world scenarios.

Conclusion

Inverse Scattering Problems lie at the heart of advanced imaging techniques, bridging the gap between wavefield measurements and meaningful reconstructions. While challenges persist, the integration of traditional methods with machine learning offers a transformative pathway for solving ISPs in complex and noisy environments. These innovations hold immense promise, particularly in the realm of biomedical imaging, where early and accurate detection can save countless lives.



CHAPTER 3 METHODOLOGY

Methodology

A 2-D transverse-magnetic scenario is considered in the present work [10]. In particular, N_i line sources illuminate a domain of interest D , where electromagnetic waves interact with scatterers. The scattered field is then collected at surface S by N_r antennas. (see scenario at the left side of Fig. 1).

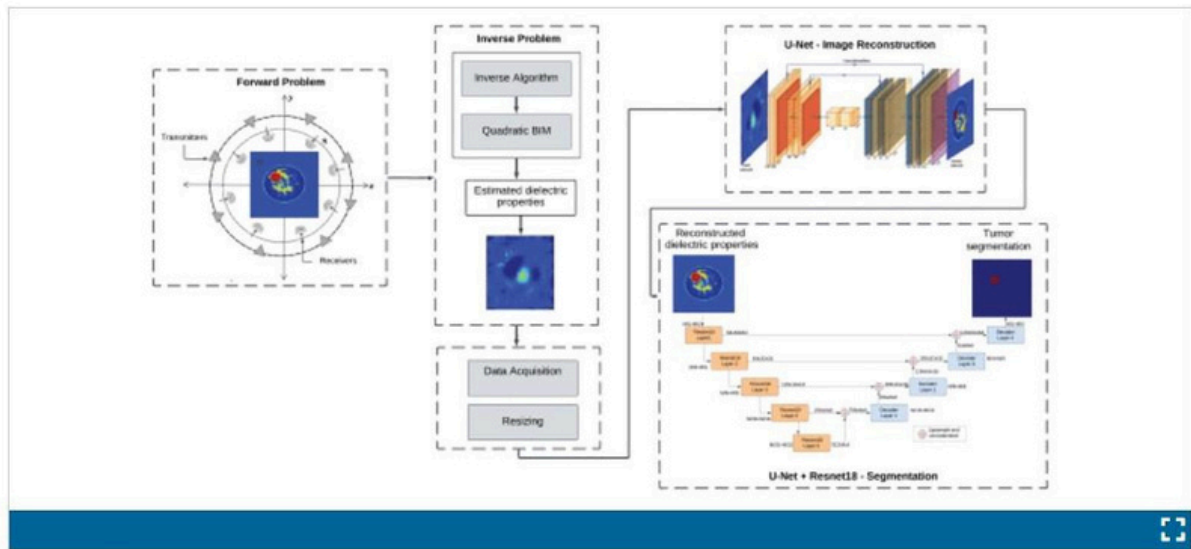


FIGURE 1.
General framework of the proposed methodology.

A BIM strategy is considered which optimizes the inverse subproblem using Quadratic Programming approach [30], making it possible to employ an exact methodology with an appropriate mechanism to enforce a specific type of *a priori* information, as the upper and lower bounds of the range of contrast values for the object to be reconstructed. In the adopted method, the optimization is performed by minimizing the residual error between the measured scattered field $E_{sm}^{\sim}$ and the computed scattered field E_{sm} [26], [27], [28], namely:

$$\min_{\{\chi_n\}} \sum_{m=1}^M \left| \widetilde{E_m^s} - \sum_{n=1}^N g_{mn} \chi_n E_n^t \right|^2$$

$$\chi_n \in \mathbb{C}, \quad n = 1, \dots, N \quad (1)$$

► View Source ?

where:

$$g_{mn} = -\frac{j}{2} \pi k_b a_n J_1(k_b a_n) H_0^{(2)}(k_b |r_m - r_n|) \quad (2)$$

► View Source ?

is the discretized version of the Green's function, and:

- $a_n = \sqrt{\Delta x \Delta y / \pi}$;
- r_n is the vector position of the n -th pixel in D ;
- $J_1(\cdot)$ is the Bessel function of the first kind;
- $H_0^{(2)}(\cdot)$ is the Hankel function of the second kind of order 0;
- $k_b = \omega \sqrt{\varepsilon_b \mu_0}$ is the wavenumber of the homogeneous medium background with relative permittivity ε_b ;
- χ_n is the contrast value at r_n , which is defined as: $\chi(r) = \frac{\varepsilon_r(r)}{\varepsilon_{rb}} - 1 - j \frac{(\sigma(r) - \sigma_b)}{\omega \varepsilon_b}$
- E_n^t is the total electric field at position r_n .

The Born Iterative Method (BIM): A Pillar of Inverse Scattering Problems

1. Introduction to the Born Iterative Method (BIM)

The Born Iterative Method (BIM) is a widely adopted technique for solving inverse scattering problems (ISP), particularly in the context of electromagnetic wave interactions with heterogeneous media. By iteratively refining the solution, BIM addresses the complexities of reconstructing the dielectric properties of a medium. Its iterative nature and relatively straightforward implementation have made BIM a cornerstone for applications like biomedical imaging, geophysics, and nondestructive testing.

2. The Fundamental Assumption: The Born Approximation

The BIM relies on the Born approximation, which simplifies the scattering problem by assuming that the scattered field is linearly proportional to the contrast between the scatterer and the surrounding medium. While this assumption holds for weak scatterers where the contrast is small, it becomes less accurate for strong scatterers, such as dense tumors in breast imaging. This limitation necessitates iterative refinements to account for the nonlinearities introduced by stronger scatterers.

3. Mathematical Framework of BIM

In the BIM, the reconstruction process begins with an initial estimate of the medium's contrast profile. The scattered field E_s is expressed as a function of the contrast χ :

$$E_s = \int g(r, r') \chi(r') E_t(r') dr' \quad E_s = \int g(r, r') \chi(r') E_t(r') dr'$$

Here:

- $g(r, r')$ is the Green's function, representing the wave's propagation in the background medium.
- $\chi(r')$ is the contrast, defined as the relative difference in permittivity and conductivity between the object and the background.
- $E_t(r')$ is the total electric field.

Using this model, BIM iteratively updates the contrast estimate by minimizing the residual error between the measured scattered field and the computed scattered field.

4. Iterative Refinement in BIM

The iterative process in BIM involves two key steps in each iteration:

1. Forward Problem Solution: The total electric field is calculated using the current estimate of the contrast.

2. Inverse Problem Update: The contrast profile is updated by minimizing the error between the measured and computed scattered fields.

The optimization process can incorporate regularization techniques to handle noise and stabilize the solution, especially in ill-posed scenarios.

5. BIM in Microwave Imaging

In microwave imaging, BIM is used to reconstruct the dielectric properties of tissues. This is critical for identifying abnormalities such as tumors, which exhibit higher permittivity and conductivity than surrounding healthy tissue. The iterative nature of BIM ensures that the reconstructions improve with each iteration, providing increasingly accurate representations of the internal tissue structure.

6. Advantages of BIM

BIM offers several advantages, which have contributed to its widespread adoption:

- **Simplicity:** Its algorithmic structure is straightforward and easy to implement.
- **Iterative Improvement:** BIM refines the solution with each iteration, improving accuracy over time.
- **Compatibility:** It integrates well with regularization techniques to mitigate the effects of noise and ill-posedness.

7. Limitations of BIM

Despite its strengths, BIM has notable limitations:

- **Sensitivity to Strong Scatterers:** The Born approximation, which underpins BIM, assumes weak scattering. In cases involving strong scatterers, such as dense tumors, the method may fail to converge or produce inaccurate results.
- **Computational Intensity:** Each iteration requires solving the forward problem, leading to high computational costs, especially for large datasets or high-resolution reconstructions.
- **Dependence on Initial Estimates:** The accuracy of the reconstruction is heavily influenced by the quality of the initial guess, making it challenging to achieve reliable results in complex scenarios.

8. Enhancing BIM with Quadratic Programming

Recent advancements have sought to enhance BIM's performance using quadratic programming techniques. By framing the inverse problem as a quadratic optimization problem,

these approaches introduce constraints, such as bounds on the contrast values, to guide the solution toward physically realistic outcomes. This refinement ensures greater accuracy and stability, even in the presence of strong scatterers.

9. Integration with Machine Learning

One transformative enhancement to BIM is its integration with machine learning.

Convolutional neural networks (CNNs) can be used to refine the reconstructions obtained from BIM, overcoming the limitations of the Born approximation. For instance:

- **Feature Extraction:** CNNs extract high-level features from preliminary reconstructions, enhancing accuracy.
- **Noise Resilience:** Machine learning models trained on noisy datasets can mitigate the effects of measurement noise.
- **Reduced Iterations:** By providing a better initial guess, CNNs reduce the number of iterations required for convergence, lowering computational costs.

10. BIM in the Context of Breast Cancer Detection

In breast cancer detection, BIM plays a critical role in reconstructing high-contrast dielectric profiles of breast tissues. Tumors, being strong scatterers, challenge the Born approximation, but iterative refinements enable reasonably accurate reconstructions. The integration of machine learning further enhances this process, enabling precise localization and characterization of tumors, even in noisy environments.

11. Noise Handling in BIM

Real-world applications often involve noise in the measured data. BIM addresses this challenge through regularization techniques, such as Tikhonov regularization or total variation regularization. These methods suppress noise-induced artifacts, ensuring stable and accurate reconstructions.

12. Computational Cost Considerations

The computational cost of BIM is a critical factor, especially for large-scale problems.

Advances in parallel computing and GPU acceleration have significantly reduced the runtime of BIM-based reconstructions. Hybrid approaches that combine BIM with CNNs also offer computational savings by leveraging pre-trained networks for faster convergence.

13. Numerical Validation Using Breast Phantoms

Datasets like the UWCEM breast phantom repository are invaluable for validating BIM-based methods. These phantoms provide realistic models of breast tissues, enabling researchers to evaluate the performance of BIM under controlled conditions. Numerical results demonstrate that while BIM can achieve high accuracy, its performance improves significantly when combined with machine learning.

14. Applications Beyond Biomedical Imaging

While this report focuses on biomedical imaging, BIM is also widely used in other domains, such as geophysics for subsurface imaging, nondestructive testing for defect detection, and through-the-wall imaging for security applications. The versatility of BIM underscores its importance across multiple fields.

15. Future Directions for BIM

Ongoing research aims to further enhance BIM's accuracy and efficiency. Emerging trends include:

- **Physics-Informed Machine Learning:** Incorporating physical constraints directly into neural networks to improve generalization.
- **Adaptive Regularization:** Developing adaptive methods that dynamically adjust regularization parameters based on the reconstruction quality.
- **Quantum Computing:** Exploring quantum algorithms for solving the forward and inverse problems, potentially reducing computational costs exponentially.

Conclusion

The Born Iterative Method remains a cornerstone of inverse scattering problems, providing a robust framework for reconstructing complex media. While its limitations in handling strong scatterers and computational intensity are well-known, advancements such as quadratic programming and machine learning integration are addressing these challenges. In biomedical imaging, particularly breast cancer detection, BIM continues to play a pivotal role, offering a pathway to accurate, non-invasive diagnostics.

CHAPTER 4: CNN

As shown in Fig. 1, a Machine Learning model is performed to overcome the limitations of BIM, thus enhancing the reconstruction of dielectric profiles. In order to accomplish for this, CNN is based on the U-net [19], which is initially created for segmentation. Several specific features of its architecture make it suitable for our goals. For this procedure, images from the breast imaging dataset estimated by the BIM are resized for use as input to the CNN. The strength of the U-Net architecture is related to its ability to identify unclear and jagged discontinuities and boundaries, otherwise strongly difficult to be detected with conventional methods, especially regarding tumor region detection issues. In this paper, three CNN architectures are discussed, namely:

1. The first architecture is introduced in our previously published works [26], [27], [28] and reported in Fig. 3. This network is based on a (CNN) and consists of an encoding and a decoding part. The input image from the network is of size 128×128 pixels. The contracting path employs two 3×3 convolutional layers with stride 1, containing 128 filters of size 128×128 ; each convolution is followed by batch normalization and rectified linear unit (ReLU). Next, a max pooling operation of 2×2 with stride 2 is applied; in this case, the filters are doubled to 256, and the size of the feature maps is reduced to 64×64 . The lower level consists of two 3×3 convolutional layers and a 2×2 max pooling layer; consequently, the number of filters is increased to 512, and the size of the feature map is reduced to 32×32 . The expansive path can resample the feature maps from the bottom up to recover the original dimensions of the input images, by means of a 2×2 up-sampling layer followed by two 3×3 convolutions, with batch normalization and rectified linear unit (ReLU). In this instance, the filters are reduced from 256 to 128, and the size of the feature maps is doubled to 64×64 . The same process is repeated at the next level of the expansive path of the network, yielding 128 filters and a dimension of 128×128 pixels. In the expansive path there are also two concatenations with the corresponding feature maps of the contractive path, which allows extracting the most important features of the previous layers and joining them to the current layers. The final layer is a 1×1 convolution to map the feature vector to the final prediction.

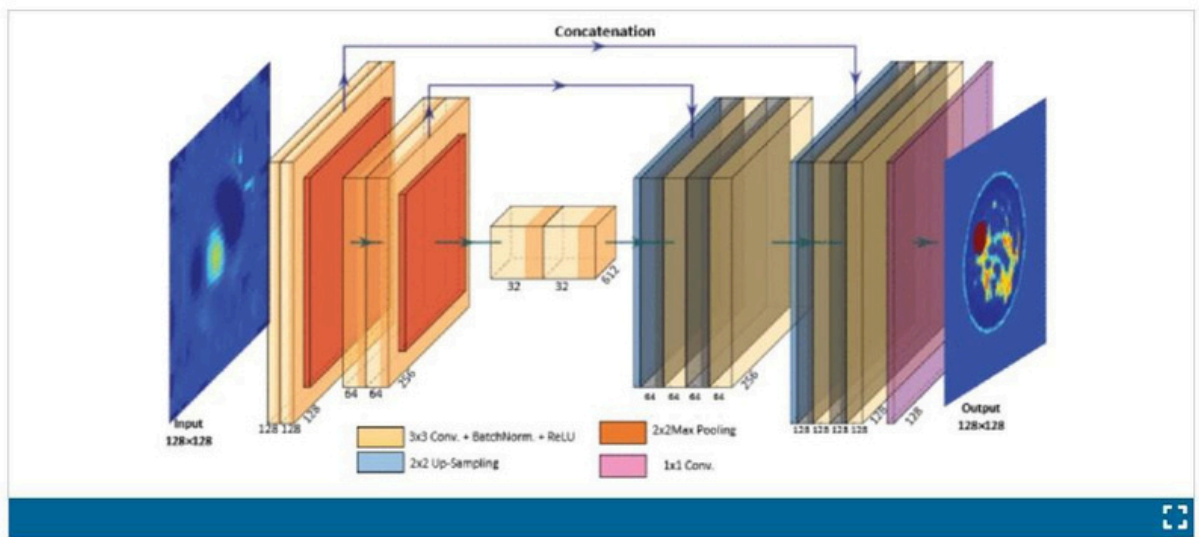


FIGURE 3.
Illustration of the U-net architecture 1.

2. Regarding the second architecture, depicted in Fig. 4, a modification is performed to the previous network. In particular, one convolution level is increased, and the number of filters in each layer is halved from 128 to 64.

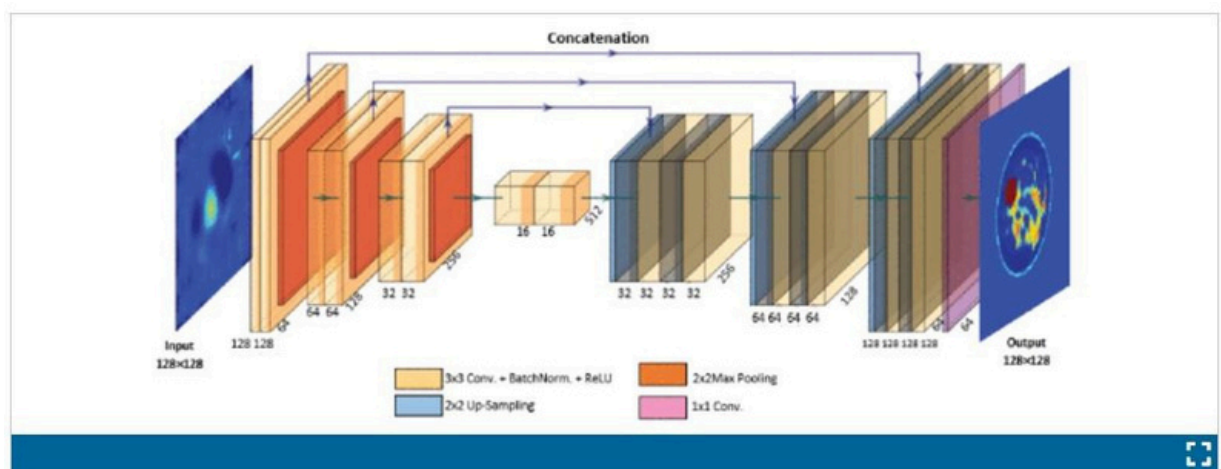


FIGURE 4.
Illustration of the U-net architecture 2.

3. The third architecture, illustrated in Fig. 5, maintains the same number of network levels as in case 1, but only reducing the number of filters from 128 to 64.

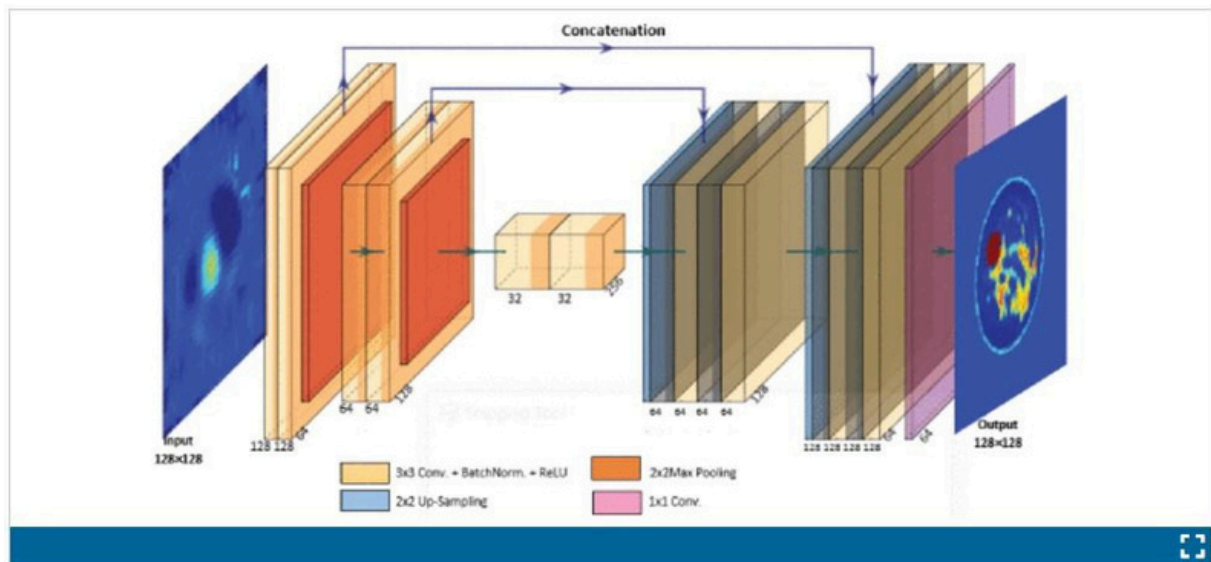
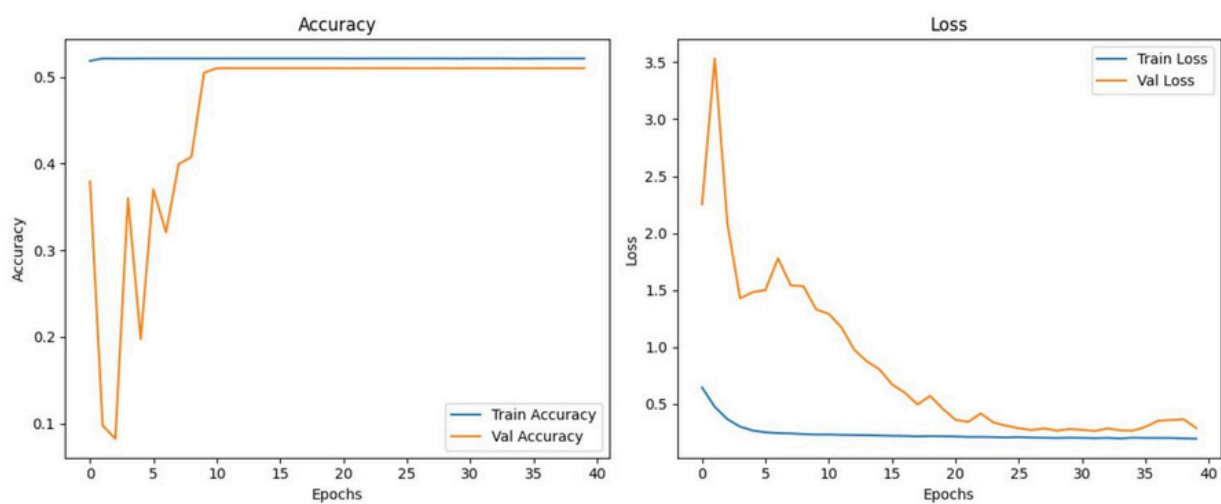
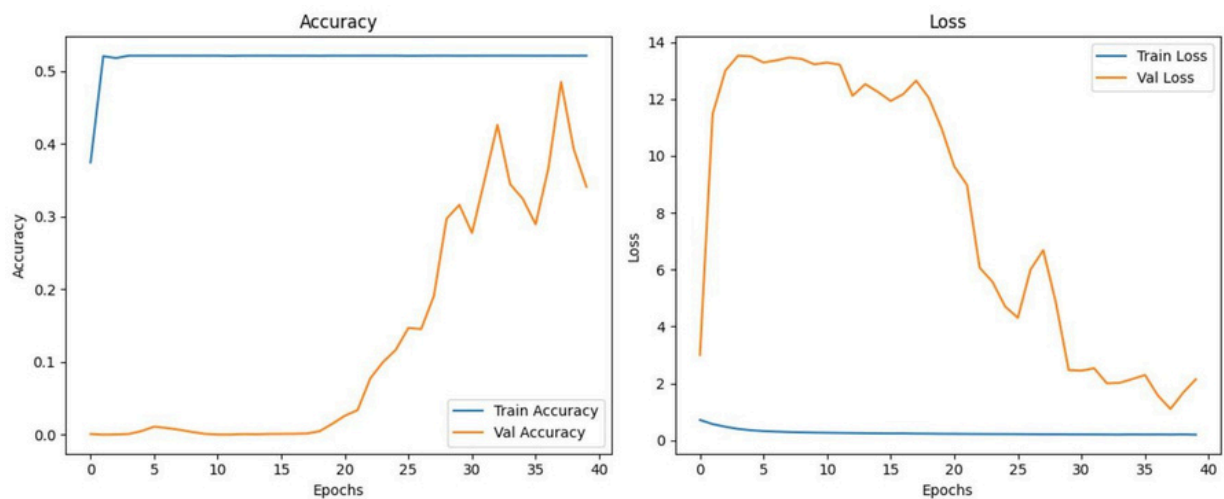


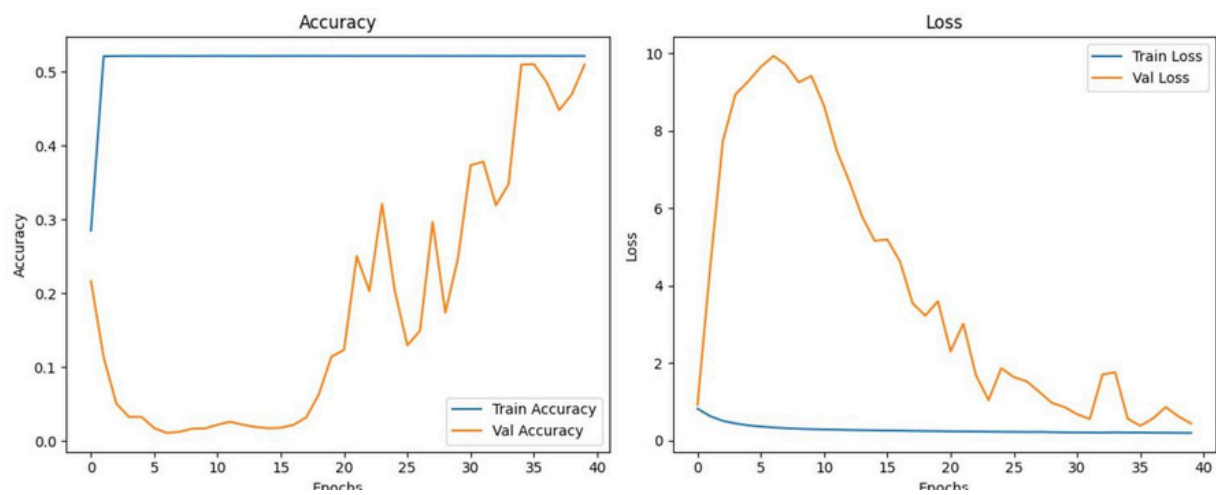
FIGURE 5.
Illustration of the U-net architecture 3.



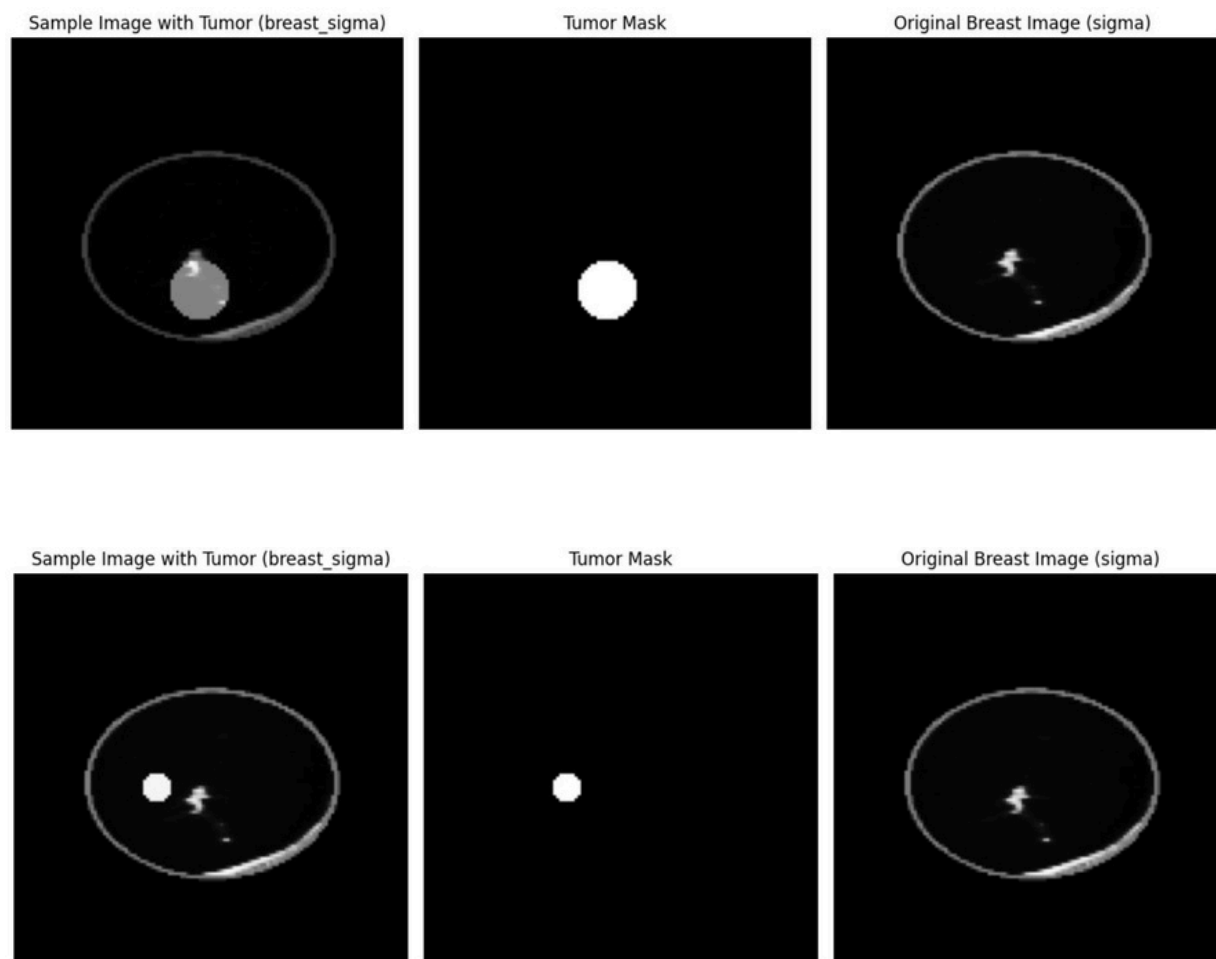
Reconstruction 1 Accuracy and Loss



Reconstruction 2 Accuracy and Loss



Reconstruction 3 Accuracy and Loss



CHAPTER 5 Tumor Segmentation

TUMOR Segmentation

The segmentation process of the images obtained from each U-Net architecture is incorporated into our proposed approach to enhance the precision of breast tumor detection and identification. The segmentation task employs an extended version of the U-Net architecture, which integrates ResNet-18 as a backbone for feature extraction. The dielectric profiles reconstructed in the preceding step are used as input for the network, while the tumor masks represent the output, as depicted in Figure 6. In this case, the ResNet-18 architecture is applied instead of the traditional U-Net encoder path. Specifically, the ResNet-18 [32], [33] model is a convolutional neural network characterized by its deep architecture and residual connections. The structure is composed by 18 layers, which include residual blocks. The utilization of residual connections enables the direct propagation of gradients from subsequent layers to earlier layers, thereby facilitating the learning process of the network and mitigating the issue of fading gradients. The utilization of residual connections within ResNet-18 facilitates the acquisition and retention of pertinent features throughout the encoding procedure. The aforementioned characteristics can be subsequently employed by the decoder path to achieve precise segmentation.

Segmentation Methodology U-Net and ResNet Integration for Segmentation

This project employs a hybrid architecture combining U-Net and ResNet-18 for segmentation. U-Net provides the symmetric encoder-decoder structure needed for precise boundary localization, while ResNet-18 enhances feature extraction through its residual learning framework.

The Encoder: ResNet-18 as a Backbone

The encoder in this hybrid model is built on ResNet-18, which extracts hierarchical features from the input dielectric profiles. Key aspects of ResNet-18 include:

- **Residual Connections:** These connections help preserve feature integrity by allowing gradients to flow through the network without vanishing.
- **Downsampling:** Successive layers reduce the spatial dimensions, focusing on high-level, abstract features.

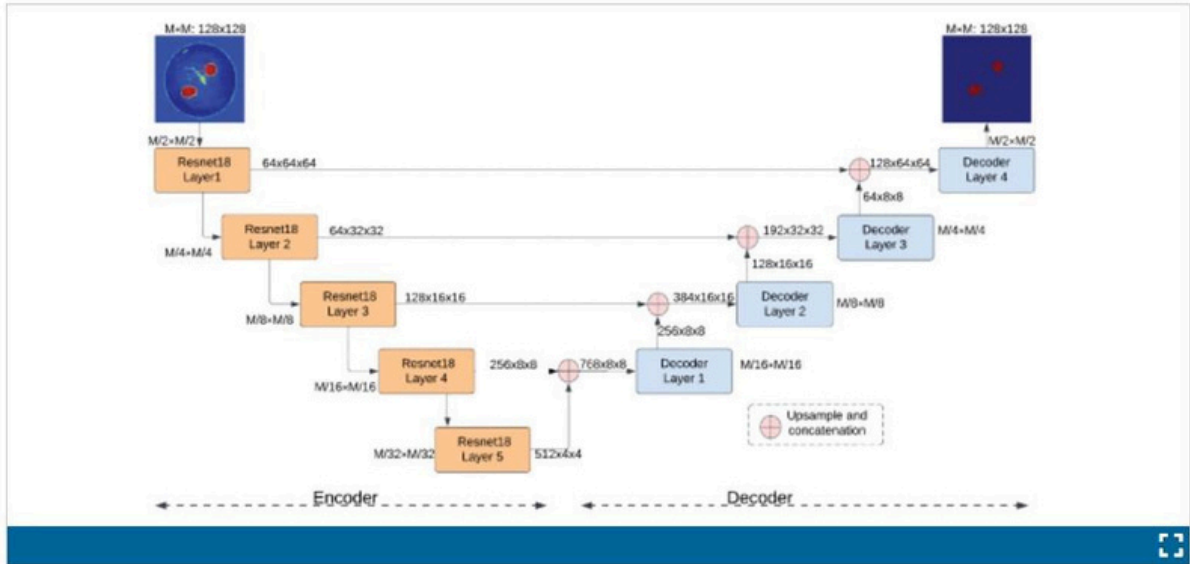


FIGURE 6.

U-Net architecture, which incorporates a ResNet18 backbone for tumor segmentation.

In the following, the individual components of the proposed segmentation network is described in detail.

- Encoder Path (Resnet 18):

The input to the network is a single-channel image with a dimension of $M \times M$ (128×128) pixels. The initial layer (Layer 1) includes a convolutional layer with a kernel size of 7×7 , featuring 64 filters, a stride of 2, and a padding of 3. Batch normalization is implemented on the output of the convolutional layer, followed by the Rectified Linear Unit (ReLU) application as the activation function. Then, a max pooling operation is executed with a kernel size of 3×3 , a stride of 2, and a padding of 1. ResNet-18 comprises residual blocks, namely layer2, layer3, layer4, and layer5, each encompassing two basic blocks. The basic blocks consist of a pair of 3×3 convolutional layers, each with 64, 128, 256, and 512 filters, respectively. The utilization of batch normalization and ReLU activation is implemented after every convolutional layer. The downsampling process is executed within the layer3, layer4, and layer5 blocks through convolutional layers featuring a stride value of 2. In addition, skip connections are established within the downsampling blocks to concatenate feature maps originating from the encoder path to those on the decoder path.

Decoder Path:

The part responsible for decoding includes a sequence of three deconvolutional blocks. Each deconvolutional block executes upsampling through the utilization of transpose convolution with a kernel size of 2×2 and a stride of 2. Each block also includes a pair of convolutional layers which use a kernel size of 3×3. After each convolutional layer, Batch Normalization and ReLU activation functions are implemented.

The output block executes the final upsampling process by utilizing transpose convolution. The architecture comprises a convolutional layer with a kernel size of 3×3 and 64 channels. The convolutional layer is followed by applying Batch Normalization and ReLU activation. The network output is achieved by employing a 1×1 convolutional layer which produces two output channels, corresponding to the number of segmentation classes.

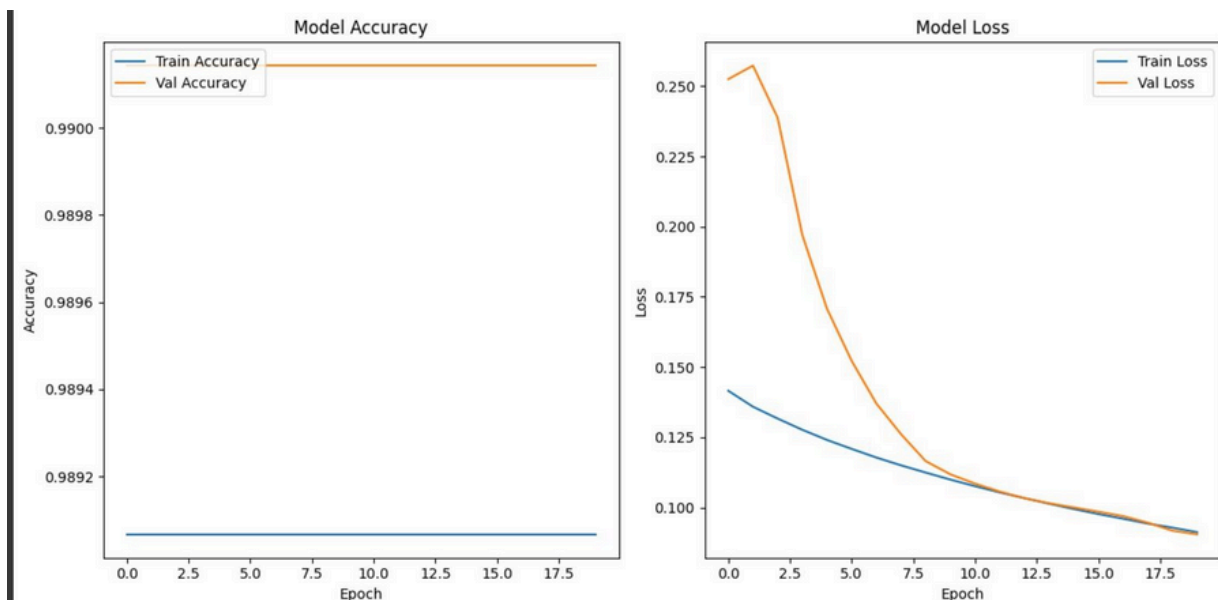
The involved network optimization process adopts the loss function Binary cross-entropy (BCE) with logits loss [34]. This function applies the sigmoid activation function to the network outputs, thereby constraining them from 0 to 1. The binary cross entropy function is computed using the following equation:

$$L_{BCE} = -\frac{1}{N} \sum_{i=1}^N [p_i \ln \hat{p}_i + (1 - p_i) \ln(1 - \hat{p}_i)] \quad (3)$$

```

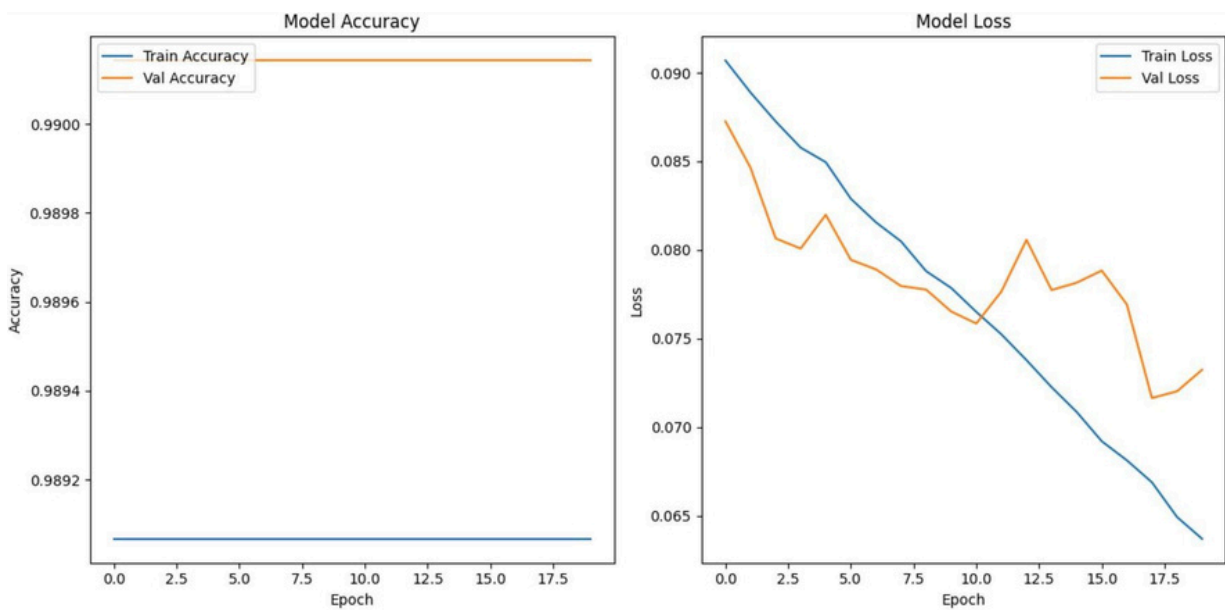
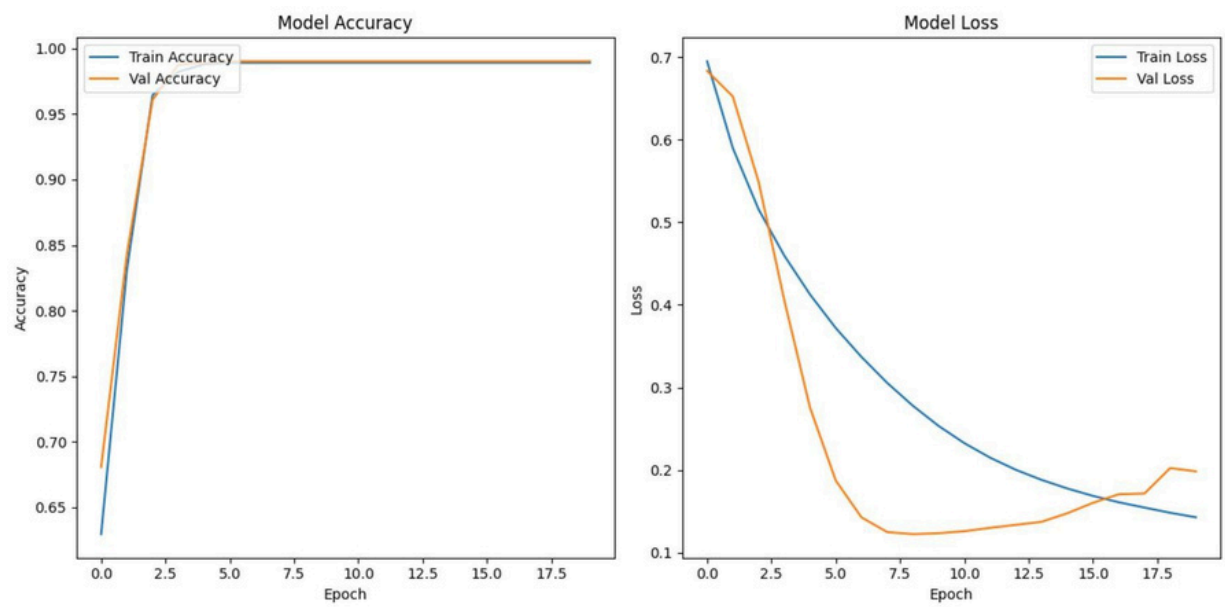
2/2 ————— 0s 133ms/step - accuracy: 0.9890 - loss: 0.0738 - val_accuracy: 0.9901 - val_loss: 0.0806
Epoch 14/20
2/2 ————— 0s 134ms/step - accuracy: 0.9892 - loss: 0.0723 - val_accuracy: 0.9901 - val_loss: 0.0777
Epoch 15/20
2/2 ————— 0s 138ms/step - accuracy: 0.9891 - loss: 0.0708 - val_accuracy: 0.9901 - val_loss: 0.0781
Epoch 16/20
2/2 ————— 0s 151ms/step - accuracy: 0.9887 - loss: 0.0697 - val_accuracy: 0.9901 - val_loss: 0.0788
Epoch 17/20
2/2 ————— 0s 145ms/step - accuracy: 0.9893 - loss: 0.0681 - val_accuracy: 0.9901 - val_loss: 0.0769
Epoch 18/20
2/2 ————— 0s 146ms/step - accuracy: 0.9896 - loss: 0.0660 - val_accuracy: 0.9901 - val_loss: 0.0716
Epoch 19/20
2/2 ————— 0s 155ms/step - accuracy: 0.9889 - loss: 0.0650 - val_accuracy: 0.9901 - val_loss: 0.0720
Epoch 20/20
2/2 ————— 0s 149ms/step - accuracy: 0.9895 - loss: 0.0627 - val_accuracy: 0.9901 - val_loss: 0.0732

```



```
Epoch 10/20
2/2 — 0s 137ms/step - accuracy: 0.9896 - loss: 0.2546 - val_accuracy: 0.9901 - val_loss: 0.1237
Epoch 11/20
2/2 — 0s 139ms/step - accuracy: 0.9897 - loss: 0.2334 - val_accuracy: 0.9901 - val_loss: 0.1261
Epoch 12/20
2/2 — 0s 134ms/step - accuracy: 0.9896 - loss: 0.2157 - val_accuracy: 0.9901 - val_loss: 0.1301
Epoch 13/20
2/2 — 0s 140ms/step - accuracy: 0.9890 - loss: 0.2016 - val_accuracy: 0.9901 - val_loss: 0.1338
Epoch 14/20
2/2 — 0s 136ms/step - accuracy: 0.9885 - loss: 0.1901 - val_accuracy: 0.9901 - val_loss: 0.1374
Epoch 15/20
2/2 — 0s 152ms/step - accuracy: 0.9888 - loss: 0.1791 - val_accuracy: 0.9901 - val_loss: 0.1478
Epoch 16/20
2/2 — 0s 146ms/step - accuracy: 0.9886 - loss: 0.1704 - val_accuracy: 0.9901 - val_loss: 0.1605
Epoch 17/20
2/2 — 0s 148ms/step - accuracy: 0.9890 - loss: 0.1618 - val_accuracy: 0.9901 - val_loss: 0.1708
Epoch 18/20
2/2 — 0s 145ms/step - accuracy: 0.9885 - loss: 0.1562 - val_accuracy: 0.9901 - val_loss: 0.1717
Epoch 19/20
2/2 — 0s 147ms/step - accuracy: 0.9892 - loss: 0.1486 - val_accuracy: 0.9901 - val_loss: 0.2025
Epoch 20/20
2/2 — 0s 149ms/step - accuracy: 0.9888 - loss: 0.1439 - val accuracy: 0.9901 - val loss: 0.1986
```

```
2/2 — 0s 135ms/step - accuracy: 0.9884 - loss: 0.1072 - val_accuracy: 0.9901 - val_loss: 0.1056
Epoch 13/20
2/2 — 0s 137ms/step - accuracy: 0.9888 - loss: 0.1041 - val_accuracy: 0.9901 - val_loss: 0.1033
Epoch 14/20
2/2 — 0s 136ms/step - accuracy: 0.9891 - loss: 0.1014 - val_accuracy: 0.9901 - val_loss: 0.1015
Epoch 15/20
2/2 — 0s 137ms/step - accuracy: 0.9894 - loss: 0.0989 - val_accuracy: 0.9901 - val_loss: 0.1000
Epoch 16/20
2/2 — 0s 137ms/step - accuracy: 0.9889 - loss: 0.0982 - val_accuracy: 0.9901 - val_loss: 0.0985
Epoch 17/20
2/2 — 0s 138ms/step - accuracy: 0.9895 - loss: 0.0952 - val_accuracy: 0.9901 - val_loss: 0.0969
Epoch 18/20
2/2 — 0s 134ms/step - accuracy: 0.9889 - loss: 0.0948 - val_accuracy: 0.9901 - val_loss: 0.0946
Epoch 19/20
2/2 — 0s 132ms/step - accuracy: 0.9890 - loss: 0.0931 - val_accuracy: 0.9901 - val_loss: 0.0917
Epoch 20/20
2/2 — 0s 137ms/step - accuracy: 0.9890 - loss: 0.0914 - val accuracy: 0.9901 - val loss: 0.0904
```



Chapter 6 SCOPE:

The project explores microwave imaging for breast cancer detection, emphasizing its non-invasive, non-ionizing benefits. Traditional methods like mammography often struggle with dense breast tissue and radiation risks. By integrating the Born Iterative Method (BIM) with advanced deep learning models (U-Net and ResNet), this approach aims to improve tumor segmentation, noise handling, and early detection, especially for small or subtle tumors.

Technological Scope:

Microwave imaging reconstructs the dielectric properties of tissues, leveraging the contrast between healthy and tumor tissues. BIM addresses inverse scattering, while U-Net+ResNet enhances segmentation accuracy, making this approach valuable for dense and heterogeneous tissues.

Clinical Applications:

The hybrid model promises real-time tumor detection, portable device integration, and applicability in resource-limited settings. It could complement existing modalities like MRI or mammography, offering improved accuracy and faster results.

Limitations:

Challenges include detecting very small tumors in dense tissue, ensuring generalization across diverse populations, and reliance on high-quality datasets. Future work must refine models and expand datasets for better performance.

Future Directions:

- Multimodal Imaging: Combining microwave data with MRI or CT for enhanced accuracy.
- Portable Devices: Development of lightweight tools for point-of-care applications.
- Attention Mechanisms: To improve small tumor detection and segmentation.

Broader Applications:

Beyond breast cancer, microwave imaging can detect other tumors (lung, liver, brain), characterize tissues for neurological and cardiovascular conditions, and serve in environmental monitoring (e.g., soil moisture) and geophysical imaging (e.g., subsurface structures). Military and security applications include through-the-wall imaging for rescue and surveillance operations.

This approach demonstrates a potential paradigm shift in cancer detection and imaging, blending advanced machine learning with innovative imaging technologies.

CONCLUSION:

Inverse electromagnetic scattering for breast imaging is a typical example of high contrast inversion, and the implementation required to achieve a high resolution can impose very high computational cost. To solve this inverse problem, while guaranteeing and increasing the accuracy of breast image reconstruction obtained from the quadratic BIM method, three CNN configurations based on the U-Net architecture have been explored and compared in this work, starting from a basic configuration presented in our recent preliminary works. Furthermore, after the breast profile reconstruction, a segmentation procedure is implemented to detect and remark the boundaries of the tumors present in the breast tissue. This process employs a novel architecture which integrates U-Net and Resnet18 to exploit the residual connections of ResNet- 18 and its pretrained weights, thus resulting into accurate segmentations at a lower expense. Using a collection of 1500 breast images with tumors in random places, training and validation samples for each proposed network have been generated. To analyze and validate the performance, sixty images distinct from the training set have been employed, considering first images without noise, and then images with added Gaussian noise. In addition, a detailed comparison in terms of computing costs and image reconstruction precision has been conducted, with the aim to perform the lower computational cost, while guaranteeing the convergence to an accurate solution, even in the presence of strong scatterers, thus leading the machine learning procedure to work as a regularization process overcoming the induced non-linearity. Numerical tests performed in noisy as well as noise-free environments have demonstrated that the proposed method can effectively reconstruct the distribution of the dielectric properties for the breast, with a very good ability to detect abnormal scatterers such as tumors. Nevertheless, some critical issues have been identified in relation to the breast model of class 3 and a noise level of 10 dB, where both the complexity and high density of the model may prevent an accurate reconstruction. Further studies will be performed to face and hopefully overcome the above aspects.

In terms of computational costs, the U-Net 2 and U-Net 3 architectures have revealed the fastest computing speeds, thus demonstrating the strong enhancement as compared to the initial architecture by the same authors. Additional studies are being currently developed to explore innovative CNNs with improved performance in terms of training and testing times.

REFERENCES

Research Paper References:

1. T. Deisboeck and J. Y. Kresh, "Biological scaling and physiological time: Biomedical applications" in *Complex Systems Science in Biomedicine*, New York, NY, USA: Springer, 2006.
2. R. Chen, Z. Wei, and X. Chen, "Three-dimensional through-wall imaging: Inverse scattering problems with an inhomogeneous background medium", *Proc. IEEE 4th Asia-Pacific Conf. Antennas Propag. (APCAP)*, pp. 505-506, Jun. 2015.
3. H. Kagiwada, R. Kalaba, S. Timko, and S. Ueno, "Associate memories for system identification: Inverse problems in remote sensing", *Math. Comput. Model.*, vol. 14, pp. 200-202, Jan. 1990.
4. R. Persico, "Introduction to GPR prospecting" in *Introduction to Ground Penetrating Radar: Inverse Scattering and Data Processing*, Hoboken, NJ, USA: Wiley, 2014.
5. X. Chen, "Reconstructing dielectric scatterers" in *Computational Methods for Electromagnetic Inverse Scattering*, Hoboken, NJ, USA: Wiley, 2018.
6. F. Gao, B. D. Van Veen, and S. C. Hagness, "Sensitivity of the distorted born iterative method to the initial guess in microwave breast imaging", *IEEE Trans. Antennas Propag.*, vol. 63, no. 8, pp. 3540-3547, Aug. 2015.
7. P. M. V. D. Berg, A. L. V. Broekhoven, and A. Abubakar, "Extended contrast source inversion", *Inverse Problems*, vol. 15, no. 5, pp. 1325-1344, Oct. 1999.
8. P. M. V. D. Berg and R. E. Kleinman, "A contrast source inversion method", *Inverse Problems*, vol. 13, no. 6, pp. 1607-1620, Dec. 1997.
9. X. Chen, "Subspace-based optimization method for solving inverse-scattering problems", *IEEE Trans. Geosci. Remote Sens.*, vol. 48, no. 1, pp. 42-49, Jan. 2010.
10. Z. Wei and X. Chen, "Deep-learning schemes for full-wave nonlinear inverse scattering problems", *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 4, pp. 1849-1860, Apr. 2019.
11. H. Zhang, Y. Chen, T. J. Cui, F. L. Teixeira, and L. Li, "Probabilistic deep learning solutions to electromagnetic inverse scattering problems using conditional renormalization group flow", *IEEE Trans. Microw. Theory Techn.*, vol. 70, no. 11, pp. 4955-4965, Nov. 2022.

12. M. T. McCann, K. H. Jin, and M. Unser, "Convolutional neural networks for inverse problems in imaging: A review", *IEEE Signal Process. Mag.*, vol. 34, no. 6, pp. 85-95, Nov. 2017.
13. Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning", *Nature*, vol. 521, no. 7553, pp. 436-444, 2015.
14. Y. S. Han, J. Yoo, and J. C. Ye, "Deep residual learning for compressed sensing CT reconstruction via persistent homology analysis", arXiv:1611.06391, Nov. 2016.
15. K. H. Jin, M. T. McCann, E. Froustey, and M. Unser, "Deep convolutional neural network for inverse problems in imaging", *IEEE Trans. Image Process.*, vol. 26, no. 9, pp. 4509-4522, Sep. 2017.
16. P. C. Tripathi and S. Bag, "CNN-DMRI: A convolutional neural network for denoising of magnetic resonance images", *Pattern Recognit. Lett.*, vol. 135, pp. 57-63, Jul. 2020.
17. G. Chen, P. Shah, J. Stang, and M. Moghaddam, "Learning-assisted multimodality dielectric imaging", *IEEE Trans. Antennas Propag.*, vol. 68, no. 3, pp. 2356-2369, Mar. 2020.
18. L. Li, L. G. Wang, F. L. Teixeira, C. Liu, A. Nehorai, and T. J. Cui, "DeepNIS: Deep neural network for nonlinear electromagnetic inverse scattering", *IEEE Trans. Antennas Propag.*, vol. 67, no. 3, pp. 1819-1825, Mar. 2019.
19. O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional networks for biomedical image segmentation", *Proc. Int. Conf. Med. Image Comput. Comput.-Assist. Intervent.*, pp. 234-241, 2015.
20. J. Long, E. Shelhamer, and T. Darrell, "Fully convolutional networks for semantic segmentation", *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, pp. 3431-3440, Jun. 2015.
21. Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition", *Proc. IEEE*, vol. 86, no. 11, pp. 2278-2324, Nov. 1998.
22. A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks", *Commun. ACM*, vol. 60, no. 6, pp. 84-90, May 2017.
23. K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition", *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, pp. 770-778, Jun. 2016.
24. M. Mardani, H. Monajemi, V. Pappas, S. Vasanawala, D. Donoho, and J. Pauly, "Recurrent generative adversarial networks for proximal learning and automated compressive image recovery", arXiv:1711.10046, Nov. 2017.
25. P. Putzky and M. Welling, "Recurrent inference machines for solving inverse problems", arXiv:1706.04008, Jun. 2017.

26. S. Costanzo, A. Flores, and G. Buonanno, "Machine learning approach to quadratic programming-based microwave imaging for breast cancer detection", *Sensors*, vol. 22, no. 11, pp. 4122, May 2022.
27. S. Costanzo and A. Flores, "Enhanced machine learning approach for accurate and fast resolution of inverse scattering problem in breast cancer detection", *Electronics*, vol. 11, no. 15, pp. 2308, Jul. 2022.
28. S. Costanzo, A. Flores, and G. Buonanno, "Machine learning methods for microwave imaging in cancer detection", *Proc. IEEE Int. Conf. Dependable Autonomic Secure Comput. Int. Conf. Pervasive Intell. Comput. Int. Conf. Cloud Big Data Comput. Int. Conf. Cyber Sci. Technol. Congr. (DASC/PiCom/CBDCom/CyberSciTech)* pp. 1-5, Sep. 2022.
29. M. J. Burfeindt, T. J. Colgan, R. O. Mays, J. D. Shea, N. Behdad, B. D. Van Veen, et al., "MRI-derived 3-D-printed breast phantom for microwave breast imaging validation", *IEEE Antennas Wireless Propag. Lett.*, vol. 11, pp. 1610-1613, 2012.
30. A. C. Batista, L. S. Batista, and R. Adriano, "A quadratic programming approach for microwave imaging", *IEEE Trans. Antennas Propag.*, vol. 69, no. 8, pp. 4923-4934, Aug. 2021.
31. D. S. Jones, *Acoustic and Electromagnetic Waves*, London, U.K.: Oxford Univ. Press, 1986.
32. M. Siam, M. Gamal, M. Abdel-Razek, S. Yogamani, and M. Jagersand, "RTSeg: Real-time semantic segmentation comparative study", *Proc. 25th IEEE Int. Conf. Image Process. (ICIP)*, pp. 1603-1607, Oct. 2018.
33. A. Ebrahimi, S. Luo, and R. Chiong, "Introducing transfer learning to 3D ResNet-18 for Alzheimer's disease detection.

APPENDIX A

The complete code implemented for the CNN Model:

Preprocessing:

```
# Specify the folder path in your Drive
load_path = '/content/drive/My Drive/'

# Load files from Drive
solution_epsilon_r = np.load(load_path + 'solution_epsilon_r.npy')
solution_sigma = np.load(load_path + 'solution_sigma.npy')
breast_epsilon_r = np.load(load_path + 'breast_epsilon_r.npy')
breast_sigma = np.load(load_path + 'breast_sigma.npy')

sol_epsilon_r = np.load(load_path + 'solution_epsilon_r.npy')
sol_sigma = np.load(load_path + 'solution_sigma.npy')
br_epsilon_r = np.load(load_path + 'breast_epsilon_r.npy')
br_sigma = np.load(load_path + 'breast_sigma.npy')

import numpy as np

if np.array_equal(breast_sigma[0], br_sigma[0]): # Use np.array_equal for
element-wise comparison

    print("images are same")
else:
    print("images are different")

# Cropping breast_sigma and breast_epsilon_r
def crop_center(array, target_shape):
    z, x, y = array.shape
    cropx, cropy = target_shape[1], target_shape[2]
    startx = (x - cropx) // 2
    starty = (y - cropy) // 2
    return array[:, startx:startx + cropx, starty:starty + cropy]

breast_sigma = crop_center(breast_sigma, (40, 128, 128))
breast_epsilon_r = crop_center(breast_epsilon_r, (40, 128, 128))
# Padding solution_sigma and solution_epsilon_r
def pad_to_target(array, target_shape, pad_value):
    z, x, y = array.shape
    pad_x = (target_shape[1] - x) // 2
    pad_y = (target_shape[2] - y) // 2
    padding = ((0, 0), (pad_x, target_shape[1] - x - pad_x), (pad_y,
target_shape[2] - y - pad_y))
    return np.pad(array, padding, mode='constant',
constant_values=pad_value)

solution_sigma = pad_to_target(solution_sigma, (40, 128, 128), pad_value=0)
solution_epsilon_r = pad_to_target(solution_epsilon_r, (40, 128, 128),
pad_value=10)

# Verifying shapes
print("Shape of cropped breast_sigma:", breast_sigma.shape)
print("Shape of cropped breast_epsilon_r:", breast_epsilon_r.shape)
print("Shape of padded solution_sigma:", solution_sigma.shape)
print("Shape of padded solution_epsilon_r:", solution_epsilon_r.shape)

# Cropping br_sigma and br_epsilon_r
```

```

def crop_center(array, target_shape):
    z, x, y = array.shape
    cropx = target_shape[1]
    target_shape[2] startx = (x - cropx) // 2 starty = (y - cropx) // 2
    return array[:, startx:startx + cropx, starty:starty + cropx]

br_sigma = crop_center(br_sigma, (40, 128, 128))
br_epsilon_r = crop_center(br_epsilon_r, (40, 128, 128))
# Padding sol_sigma and sol_epsilon_r
def pad_to_target(array, target_shape, pad_value):
    z, x, y = array.shape
    pad_x = (target_shape[1] - x) // 2
    pad_y = (target_shape[2] - y) // 2
    padding = ((0, 0), (pad_x, target_shape[1] - x - pad_x), (pad_y, target_shape[2] - y - pad_y))
    return np.pad(array, padding, mode='constant', constant_values=pad_value)

sol_sigma = pad_to_target(sol_sigma, (40, 128, 128), pad_value=0)
sol_epsilon_r = pad_to_target(sol_epsilon_r, (40, 128, 128), pad_value=10)
# Verifying shapes
print("Shape of cropped br_sigma:", br_sigma.shape)
print("Shape of cropped br_epsilon_r:", br_epsilon_r.shape)
print("Shape of padded sol_sigma:", sol_sigma.shape)
print("Shape of padded sol_epsilon_r:", sol_epsilon_r.shape)

import random
import math
import numpy as np
import matplotlib.pyplot as plt

# Function to add a tumor to the image without placing it in the background
def add_tumor(image, background_value, min_diameter, max_diameter, min_value, max_value):
    cm_to_pixels = 128 / 10.0 # Conversion from cm to pixels
    min_radius = (min_diameter / 2) * cm_to_pixels
    max_radius = (max_diameter / 2) * cm_to_pixels
    valid_regions = np.where(image != background_value)
    if valid_regions[0].size == 0:
        return image, np.zeros_like(image, dtype=bool)

    tumor_added = False
    retries = 0
    max_retries = 100
    tumor_mask = np.zeros_like(image, dtype=bool)
    while not tumor_added and retries < max_retries:
        center_y, center_x = random.choice(list(zip(valid_regions[0], valid_regions[1])))
        radius = random.uniform(min_radius, max_radius)
        y, x = np.ogrid[:image.shape[0], :image.shape[1]]
        mask = (x - center_x)**2 + (y - center_y)**2 <= radius**2 #circular
        mask banara

        if np.all(image[mask] != background_value):
            tumor_value = random.uniform(min_value, max_value)
            image[mask] = tumor_value
            tumor_mask[mask] = True
            tumor_added = True

```

```

        retries += 1

    return image, tumor_mask

# Function to add tumors to each 128x128 image in the dataset and generate
tumor masks
def add_tumors_to_images(images_sigma, images_epsilon_r, min_diameter,
max_diameter,
                        sigma_min, sigma_max, epsilon_min, epsilon_max,
background_sigma, background_epsilon):
    tumor_masks = []

    for i in range(images_sigma.shape[0]):
        # Add the same tumor to both sigma and epsilon_r images
        images_sigma[i], tumor_mask = add_tumor(images_sigma[i],
background_sigma,
sigma_min, sigma_max,
epsilon_min, epsilon_max,
min_diameter, max_diameter,
background_epsilon)

        images_epsilon_r[i][tumor_mask] = random.uniform(epsilon_min,
epsilon_max) # Apply tumor to epsilon_r using the same mask
        tumor_masks.append(tumor_mask)

    return images_sigma, images_epsilon_r, np.array(tumor_masks)

# Parameters
min_diameter_cm = 0.6
max_diameter_cm = 1.6
background_sigma = 0
background_epsilon = 10

tumor_sigma_min = 2.5 # Example range
tumor_sigma_max = 60
tumor_epsilon_r_min = 0.5
tumor_epsilon_r_max = 1.3

# Example breast image arrays (replace these with actual breast images in
your project)
images_sigma = np.copy(br_sigma) # Using `br_sigma` as the original
reference
images_epsilon_r = np.copy(br_epsilon_r)

# Add tumors to the images and generate binary masks
images_sigma, images_epsilon_r, tumor_masks =
add_tumors_to_images(images_sigma, images_epsilon_r, max_diameter_cm,
min_diameter_cm, tumor_sigma_min, tumor_sigma_max, tumor_epsilon_r_min, tumor_epsilon_r_max,
background_sigma, background_epsilon)

tumor_images_sigma = tumor_masks + breast_sigma
tumor_images_epsilon_r = tumor_masks + breast_epsilon_r
# Plot sample image with tumor and its binary mask
plt.figure(figsize=(12, 6))
# Plot the image with tumor (breast_sigma)
plt.subplot(1, 3, 1)
plt.imshow(tumor_images_sigma[0], cmap='gray')
plt.title("Sample Image with Tumor (breast_sigma)")

```

```

plt.axis('off')

# Plot the tumor mask
plt.subplot(1, 3, 2)
plt.imshow(tumor_masks[0], cmap='gray')
plt.title("Tumor Mask")
plt.axis('off')

# Plot the original breast image (for comparison)
plt.subplot(1, 3, 3)
plt.imshow(br_sigma[0], cmap='gray')
plt.title("Original Breast Image (sigma)")
plt.axis('off')

plt.tight_layout()
plt.show()

plt.subplot(1, 3, 1)
plt.imshow(tumor_images_epsilon_r[0], cmap='gray')
plt.title("Sample Image with Tumor (breast_epsilon_r)")
plt.axis('off')

plt.subplot(1, 3, 2)
plt.imshow(tumor_masks[0], cmap='gray')
plt.title("Tumor Mask")
plt.axis('off')

plt.subplot(1, 3, 3)
plt.imshow(br_epsilon_r[0], cmap='gray')
plt.title("Original Breast Image (epsilon_r)")
plt.axis('off')

plt.tight_layout()
plt.show()

```

RECONSTRUCTION CODE:

```

import tensorflow as tf
from tensorflow.keras import layers, Model
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split

# Define the convolutional block
def conv_block(x, filters):
    x = layers.Conv2D(filters, (3, 3), padding='same',
kernel_initializer='he_normal')(x)
    x = layers.BatchNormalization()(x)
    x = layers.ReLU()(x)
    x = layers.Conv2D(filters, (3, 3), padding='same',
kernel_initializer='he_normal')(x)
    x = layers.BatchNormalization()(x)
    x = layers.ReLU()(x)
    return x

# Define the third U-Net architecture
def build_third_unet(input_shape=(128, 128, 1)):
    inputs = layers.Input(input_shape)

    # Encoding Path
    enc1 = conv_block(inputs, 64)

```

```

pool1 = layers.MaxPooling2D(pool_size=(2, 2), strides=2)(enc1)

enc2 = conv_block(pool1, 128)
pool2 = layers.MaxPooling2D(pool_size=(2, 2), strides=2)(enc2)
enc3 = conv_block(pool2, 256)
pool3 = layers.MaxPooling2D(pool_size=(2, 2), strides=2)(enc3)
# Bottleneck
bottleneck = conv_block(pool3, 512)
# Decoding Path
up2 = layers.Conv2DTranspose(256, (2, 2), strides=2,
padding='same')(bottleneck)
up2 = layers.concatenate([up2, enc3])
dec2 = conv_block(up2, 256)

up1 = layers.Conv2DTranspose(128, (2, 2), strides=2,
padding='same')(dec2)
up1 = layers.concatenate([up1, enc2])
dec1 = conv_block(up1, 128)
up0 = layers.Conv2DTranspose(64, (2, 2), strides=2,
padding='same')(dec1)
up0 = layers.concatenate([up0, enc1])
dec0 = conv_block(up0, 64)
# Output layer
outputs = layers.Conv2D(1, (1, 1), activation='sigmoid')(dec0)
# Define model
model = Model(inputs, outputs)
return model

# Create the model
third_unet_model = build_third_unet()
# Generate data
X, Y = BIM_sigma, tumor_images_sigma
X_train, X_val, Y_train, Y_val = train_test_split(X, Y, test_size=0.2,
random_state=42)
# Compile the model
third_unet_model.compile(optimizer='adam', loss='binary_crossentropy',
metrics=['accuracy'])

# Train the model
history = third_unet_model.fit(
    X_train, Y_train,
    validation_data=(X_val, Y_val),
    epochs=40,
    batch_size=16
)

# Plot accuracy and loss graphs
# Accuracy Plot
plt.figure(figsize=(12, 5))

# Plot training & validation accuracy
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Train Accuracy')

```

```

plt.plot(history.history['val_accuracy'], label='Val Accuracy')
plt.title('Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()

# Loss Plot
plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Val Loss')
plt.title('Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()

plt.tight_layout()
plt.show()

```

SEGMENTATION:

```

class ResnetBlock(Model):
    """
    A standard resnet block.
    """

    def __init__(self, channels: int, down_sample=False):
        """
        channels: same as number of convolution kernels
        """
        super().__init__()

        self.__channels = channels
        self.__down_sample = down_sample
        self.__strides = [2, 1] if down_sample else [1, 1]

        KERNEL_SIZE = (3, 3)
        # use He initialization, instead of Xavier (a.k.a 'glorot_uniform'
        in Keras), as suggested in [2]
        INIT_SCHEME = "he_normal"
        self.conv_1 = Conv2D(self.__channels, strides=self.__strides[0],
                               kernel_size=KERNEL_SIZE, padding="same",
                               kernel_initializer=INIT_SCHEME)
        self.bn_1 = BatchNormalization()
        self.conv_2 = Conv2D(self.__channels, strides=self.__strides[1],
                               kernel_size=KERNEL_SIZE, padding="same",
                               kernel_initializer=INIT_SCHEME)
        self.bn_2 = BatchNormalization()
        self.merge = Add()

        if self.__down_sample:
            # perform down sampling using stride of 2, according to [1].
            self.res_conv = Conv2D(
                self.__channels, strides=2, kernel_size=(1, 1),
                kernel_initializer=INIT_SCHEME, padding="same")
            self.res_bn = BatchNormalization()

    def call(self, inputs):
        res = inputs
        x = self.conv_1(inputs)
        x = self.bn_1(x)

```



```

        x = tf.nn.relu(x)
        x = self.conv_2(x)
        x = self.bn_2(x)

        if self.__down_sample:
            res = self.res_conv(res)
            res = self.res_bn(res)

        # if not perform down sample, then add a shortcut directly
        x = self.merge([x, res])
        out = tf.nn.relu(x)
        return out

class ResNet18(Model):

    def __init__(self, num_classes, **kwargs):
        """
        num_classes: number of classes in specific classification task.
        """
        self.conv_1 = Conv2D(64, (7, 7), strides=2,
                               padding="same", kernel_initializer="he_normal")
        self.init_bn = BatchNormalization()
        self.pool_2 = MaxPool2D(pool_size=(2, 2), strides=2, padding="same")
        self.res_1_1 = ResnetBlock(64)
        self.res_1_2 = ResnetBlock(64)
        self.res_2_1 = ResnetBlock(128, down_sample=True)
        self.res_2_2 = ResnetBlock(128)
        self.res_3_1 = ResnetBlock(256, down_sample=True)
        self.res_3_2 = ResnetBlock(256)
        self.res_4_1 = ResnetBlock(512, down_sample=True)
        self.res_4_2 = ResnetBlock(512)
        self.avg_pool = GlobalAveragePooling2D()
        self.flat = Flatten()
        self.fc = Dense(num_classes, activation="softmax")

    def call(self, inputs):
        out = self.conv_1(inputs)
        out = self.init_bn(out)
        out = tf.nn.relu(out)
        out = self.pool_2(out)
        for res_block in [self.res_1_1, self.res_1_2, self.res_2_1,
self.res_2_2, self.res_3_1, self.res_3_2, self.res_4_1, self.res_4_2]:
            out = res_block(out)
        out = self.avg_pool(out)
        out = self.flat(out)
        out = self.fc(out)
        return out

import tensorflow as tf
from tensorflow.keras.layers import Conv2D, BatchNormalization, ReLU,
Concatenate, Conv2DTranspose, Input
from tensorflow.keras.models import Model

# ResNet18 Block Definition
class ResnetBlock(Model):

    def __init__(self, channels: int, down_sample=False):
        super().__init__()
        self.__channels = channels
        self.__down_sample = down_sample
        self.__strides = [2, 1] if down_sample else [1, 1]

        KERNEL_SIZE = (3, 3)
        INIT_SCHEME = "he_normal"

```

```

        self.conv_1 = Conv2D(self.__channels, strides=self.__strides[0],
                              kernel_size=KERNEL_SIZE, padding="same",
kernel_initializer=INIT_SCHEME)
        self.bn_1 = BatchNormalization()
        self.conv_2 = Conv2D(self.__channels, strides=self.__strides[1],
                              kernel_size=KERNEL_SIZE, padding="same",
kernel_initializer=INIT_SCHEME)
        self.bn_2 = BatchNormalization()
        self.merge = Concatenate()

        if self.__down_sample:
            self.res_conv = Conv2D(
                self.__channels, strides=2, kernel_size=(1, 1),
kernel_initializer=INIT_SCHEME, padding="same")
            self.res_bn = BatchNormalization()

    def call(self, inputs):
        res = inputs
        x = self.conv_1(inputs)
        x = self.bn_1(x)
        x = tf.nn.relu(x)
        x = self.conv_2(x)
        x = self.bn_2(x)

        if self.__down_sample:
            res = self.res_conv(res)
            res = self.res_bn(res)

        x = self.merge([x, res])
        out = tf.nn.relu(x)
        return out

# ResNet18 Encoder Definition
class ResNet18_Encoder(Model):
    def __init__(self):
        super(ResNet18_Encoder, self).__init__()
        self.conv_1 = Conv2D(64, (7, 7), strides=2, padding="same",
kernel_initializer="he_normal")
        self.init_bn = BatchNormalization()
        self.pool_2 = tf.keras.layers.MaxPool2D(pool_size=(2, 2), strides=2,
padding="same")
        self.res_1_1 = ResnetBlock(64)
        self.res_1_2 = ResnetBlock(64)
        self.res_2_1 = ResnetBlock(128, down_sample=True)
        self.res_2_2 = ResnetBlock(128)
        self.res_3_1 = ResnetBlock(256, down_sample=True)
        self.res_3_2 = ResnetBlock(256)
        self.res_4_1 = ResnetBlock(512, down_sample=True)
        self.res_4_2 = ResnetBlock(512)

    def call(self, inputs):
        x = self.conv_1(inputs)
        x = self.init_bn(x)
        x = tf.nn.relu(x)
        x = self.pool_2(x)

        skip1 = self.res_1_1(x)
        skip2 = self.res_2_1(skip1)
        skip3 = self.res_3_1(skip2)
        skip4 = self.res_4_1(skip3)

        return skip1, skip2, skip3, skip4

# Decoder Block Definition

```

```

def decoder_block(x, skip, filters):
    x = Conv2DTranspose(filters, kernel_size=(2, 2), strides=2,
padding="same")(x)
    skip = Conv2DTranspose(filters, kernel_size=(2, 2), strides=2,
padding="same")(skip)
    x = Concatenate()([x, skip])
    x = Conv2D(filters, (3, 3), padding="same")(x)
    x = BatchNormalization()(x)
    x = ReLU()(x)
    x = Conv2D(filters, (3, 3), padding="same")(x)
    x = BatchNormalization()(x)
    x = ReLU()(x)
    return x

def final_upsample_block(x, filters=64, num_classes=2):
    # Upsample using transpose convolution
    x = Conv2DTranspose(filters, kernel_size=(2, 2), strides=2,
padding="same")(x)

    x = Conv2D(filters, (3, 3), padding="same")(x)
    x = BatchNormalization()(x)
    x = ReLU()(x)

    # Final 1x1 convolution to produce the desired number of output channels
(num_classes)
    x = Conv2D(num_classes, (1, 1), activation="sigmoid" if num_classes == 1
else "softmax")(x)

    return x

# U-Net with ResNet18 Encoder
def build_unet_resnet18(input_shape=(128, 128, 1), num_classes=2):
    inputs = Input(shape=input_shape)

    # Encoder with ResNet18
    encoder = ResNet18_Encoder()
    skip1, skip2, skip3, skip4 = encoder(inputs)

    # Decoder path
    d1 = decoder_block(skip4, skip4, 512)
    d2 = decoder_block(d1, skip3, 256)
    d3 = decoder_block(d2, skip2, 128)
    d4 = decoder_block(d3, skip1, 64)

    # Final output block with upsampling and output layer
    outputs = final_upsample_block(d4, filters=64, num_classes=num_classes)

    # Construct and return the model
    model = Model(inputs, outputs)
    return model

# Compile the model
model = build_unet_resnet18()
model.compile(optimizer="adam", loss="binary_crossentropy",
metrics=["accuracy"])

```